
Cancri: Cross-Checkpoint Latent Relay

Low-Memory Expert Chaining via Hidden State Handoff

Anonymous Author(s)

Affiliation

Address

email

Abstract

We study *latent relay*, a mechanism for chaining language-model checkpoints by transferring intermediate hidden states across a split layer instead of communicating through decoded tokens. The method targets a narrow but practically relevant setting: two checkpoints with the same architecture, tokenizer, hidden dimension, and positional-encoding scheme. In this setting, the relay payload is a single activation tensor, e.g., 163,840 bytes for a 20-token sequence with $d_{\text{model}} = 2048$ in float32, while peak memory remains approximately one model plus the payload under sequential loading. On a Qwen3.5-2B Base $\rightarrow$ Instruct case study, same-weight relays are bitwise identical to ordinary single-model inference, and cross-checkpoint relays remain stable over short 20-token generations. The relay obtains a mean autoregressive perplexity ratio of $0.949\times$ relative to the Instruct baseline across three evaluation texts, a 47% three-way token-consistency rate over five prompts, and no observed short-horizon degeneration. These results support latent relay as a feasibility result for memory-efficient activation-space expert composition, while leaving larger-scale, longer-context, and benchmark-level validation as open work.

1 Introduction

Multi-model language-system pipelines usually communicate in token space: model A writes text and model B consumes that text as a prompt. This interface is simple and robust, but it compresses a high-dimensional continuous representation into a discrete token sequence before the receiver can use it. Mixture-of-experts (MoE) architectures (Shazeer et al., 2017; Fedus et al., 2022) avoid this bottleneck inside a single forward pass, but the experts typically have to be resident in memory together or managed by a specialized serving system.

We explore a third interface: **latent relay**. Expert A executes the first K transformer layers on an input sequence, writes the hidden state tensor to a relay buffer, and is then evicted. Expert B is loaded and continues the computation from layer K to the output head. The relay is therefore an activation-space handoff between same-architecture checkpoints rather than a token-mediated conversation. Figure 1 contrasts the two interfaces.

The work is best understood as a concept-and-feasibility study rather than a claim of general benchmark superiority. We make four contributions:

[leftmargin=*,topsep=1pt,itemsep=1pt]

1. We formalize the latent-relay operator for same-architecture transformer checkpoints and state the assumptions under which the receiver can recompute rotary positional embeddings (RoPE) independently.

Conventional: Input tokens $\rightarrow$ Checkpoint A $\rightarrow$ token decode $\rightarrow$ Checkpoint B. Latent relay: Input tokens $\rightarrow$ Checkpoint A layers $0:K \rightarrow$ hidden-state relay buffer $\rightarrow$ Checkpoint B layers $K:N$.

Figure 1: Token-mediated chaining versus latent relay. Latent relay passes an intermediate hidden-state tensor directly from checkpoint A to checkpoint B.

2. We verify a zero-error engineering baseline: when the sender and receiver weights are identical, relay execution is bitwise identical to the ordinary single-model forward pass across all split points tested.
3. We provide a small cross-checkpoint Base $\rightarrow$ Instruct case study showing stable short-horizon behavior and competitive teacher-forcing perplexity.
4. We characterize a three-zone split-point pattern and discuss the limitations, reproducibility requirements, and responsible-release considerations needed before scaling the idea.

2 Related work

Model stitching. Model stitching studies whether intermediate representations from different networks can be aligned by learned transformations (Lenc and Vedaldi, 2015; Bansal et al., 2021). Latent relay differs by testing a stricter zero-shot setting: no alignment layer is trained, and compatibility comes only from shared architecture and checkpoint lineage.

Model merging. Weight-space methods such as TIES merging (Yadav et al., 2023) and SLERP-style merging (Goddard et al., 2024) produce a single set of merged parameters. Latent relay leaves all parameters unchanged and composes checkpoints at inference time in activation space.

Speculative decoding and MoE inference. Speculative decoding (Leviathan et al., 2023) uses a draft model and a verifier for latency reduction, with token-level interaction. MoE systems route activations among expert submodules during a single forward pass (Shazeer et al., 2017; Fedus et al., 2022). Latent relay instead temporally multiplexes full checkpoints, trading additional loading latency for low peak memory.

3 Method

3.1 Setup

Let $\mathcal{M}_A$ and $\mathcal{M}_B$ be transformer language-model checkpoints sharing architecture hyperparameters $(N, d_{\text{model}}, H, d_{\text{ff}})$, tokenizer, vocabulary, and positional-encoding scheme, but with parameters $\Theta_A \neq \Theta_B$. A standard forward pass factorizes as

$$\mathbf{h}^{(0)} = \text{Embed}(\mathbf{x}; \Theta), \quad (1)$$

$$\mathbf{h}^{(i)} = \text{Layer}_i(\mathbf{h}^{(i-1)}, \rho(\mathbf{p}); \Theta), \quad i = 1, \dots, N, \quad (2)$$

$$\mathbf{y} = \text{LMHead}(\text{Norm}(\mathbf{h}^{(N)}; \Theta); \Theta), \quad (3)$$

where $\mathbf{x} \in \mathbb{Z}^L$ and $\mathbf{p} = (0, \dots, L-1)$.

Definition 1 (Latent relay). For a split layer $K \in \{1, \dots, N-1\}$, latent relay from $\mathcal{M}_A$ to $\mathcal{M}_B$ is

$$\mathbf{h}_A^{(0)} = \text{Embed}(\mathbf{x}; \Theta_A), \quad (4)$$

$$\mathbf{h}_A^{(K)} = \text{Layers}_{0:K}(\mathbf{h}_A^{(0)}, \rho_A; \Theta_A), \quad (5)$$

$$\tilde{\mathbf{h}}^{(K)} = \mathbf{h}_A^{(K)}, \quad (6)$$

$$\mathbf{h}_{A \rightarrow B}^{(N)} = \text{Layers}_{K:N}(\tilde{\mathbf{h}}^{(K)}, \rho_B; \Theta_B), \quad (7)$$

$$\mathbf{y}_{A \rightarrow B} = \text{LMHead}(\text{Norm}(\mathbf{h}_{A \rightarrow B}^{(N)}; \Theta_B); \Theta_B). \quad (8)$$

62 The transferred payload has shape $1 \times L \times d_{\text{model}}$. In float32 its size is $4Ld_{\text{model}}$ bytes; for $L = 20$
63 and $d_{\text{model}} = 2048$ this is 163,840 bytes (≈ 160 KB). In fp16 the same payload is approximately 80
64 KB. This accounting excludes model-load latency and disk-cache effects.

65 3.2 RoPE checkpoint invariance

66 **Proposition 1** (RoPE checkpoint invariance). *Let $\rho(\mathbf{p}; \theta_{\text{rope}})$ denote RoPE with hyperparameters*
67 *$\theta_{\text{rope}} = (\text{head_dim}, \text{base})$. If $\mathcal{M}_A$ and $\mathcal{M}_B$ share the same architecture specification and do not*
68 *alter RoPE hyperparameters during fine-tuning, then $\rho_A(\mathbf{p}) = \rho_B(\mathbf{p})$ for all valid position indices*
69 *$\mathbf{p}$.*

70 *Proof.* RoPE frequencies are determined by fixed architecture hyperparameters, not by learned
71 weights. Fine-tuning changes Θ_A or Θ_B but does not change `head_dim` or `base` under the stated
72 assumptions. Thus both checkpoints compute the same position-dependent rotations for the same
73 position indices. $\square$

74 **Corollary 1.** *The receiver can recompute the positional embedding locally; the relay buffer only*
75 *needs the hidden state tensor and minimal execution metadata such as the split layer and sequence*
76 *length.*

77 3.3 Memory and latency model

78 Sequential relay execution loads A, computes layers $0:K$, writes the payload, evicts A, loads B, and
79 computes layers $K:N$. The peak memory is therefore

$$M_{\text{peak}} \approx M_{\text{model}} + 4Ld_{\text{model}}, \quad (9)$$

80 for a two-checkpoint relay, assuming full eviction and no framework-level caching of the previous
81 checkpoint. For a chain of n checkpoints, peak memory remains approximately one model plus the
82 largest payload, while latency includes n checkpoint-load events and sequential computation.

83 4 Experimental setup

84 4.1 Models and hardware

85 We use a same-family Qwen3.5-2B Base and Instruct pair with 24 layers, hidden dimension 2048,
86 8 attention heads with 2 KV heads, RoPE head dimension 64, vocabulary size 248,320, and float32
87 inference. Experiments were run on an Intel Core i5-10400 CPU system with 16 GB DDR4 RAM
88 and SSD storage. An NVIDIA GT730 GPU was present but not used for reported runs. Each relay
89 stage was executed in a subprocess to make model eviction explicit.

90 4.2 Evaluation protocol

91 We report teacher-forcing perplexity,

$$\text{PPL}(\mathbf{x}) = \exp \left(-\frac{1}{L-1} \sum_{t=1}^{L-1} \log p_{\theta}(x_{t+1} \mid x_{1:t}) \right), \quad (10)$$

92 and two ratios: $\text{PPL}_{\text{relay}} / \min(\text{PPL}_A, \text{PPL}_B)$ and $\text{PPL}_{\text{relay}} / \text{PPL}_B$. We also compare generated
93 tokens over 20 autoregressive steps across five prompts: Chinese science, English philosophy, Chi-
94 nese technology, Python code, and Chinese creative fiction. The exact prompt strings and command
95 lines should be included in the anonymized supplementary code package for submission; the main
96 paper avoids author-identifying paths or repository names.

97 **Statistical scope.** The experiments use deterministic inference and a small prompt set. We there-
98 fore do not claim statistical significance or benchmark-level superiority. The empirical claims are
99 limited to implementation correctness and short-horizon feasibility under the stated checkpoint pair
100 and split settings.

Table 1: Single-step relay PPL at split $K = 6$. The relay improves on the Instruct checkpoint in all three cases, but not always on the best checkpoint.

Text	PPL _A	PPL _B	PPL _R	vs. best	vs. B
Quantum (ZH)	2.97	3.54	3.04	1.024	0.859
English	10.10	9.61	9.09	0.946	0.946
ML (ZH)	3.84	4.09	3.88	1.010	0.949
<i>Mean</i>				0.993	0.918

Table 2: Token consistency over 20-step autoregressive relay. “Neither” tokens were manually inspected and did not show short-horizon degeneration in these runs.

Category	Count	Rate
Tri-consistent (relay = A = B)	47	47.0%
B-only (relay = B $\neq$ A)	15	15.0%
A-only (relay = A $\neq$ B)	5	5.0%
Neither	33	33.0%
Aligned with B	62	62.0%

5 Results

5.1 Engineering correctness

Manual layer-by-layer forward passes reproduced the standard `model()` output with `max_diff=0.00e+00` across all 25 split points ($K = 0, 1, \dots, 24$) and all four correctness-test sequences, both for a single model instance and for disk-serialized relay across isolated subprocesses. This validates the implementation baseline: any cross-checkpoint behavior below is caused by checkpoint differences rather than relay serialization error.

5.2 Single-step cross-checkpoint relay

For split $K = 6$, Table 1 reports three teacher-forcing texts. The relay is below the Instruct PPL on all three texts and below both baselines on the English text. The mean relay-to-best ratio is 0.993, while the mean relay-to-Instruct ratio is 0.918.

Split-point trend. As K increases from 1 to 23, the L^∞ distance to B increases from 1.21 to 4.30, while the distance to A decreases from 4.74 to 3.91. This monotone trend suggests that the split layer acts as a continuous interpolation control between sender and receiver behavior for this checkpoint pair.

5.3 Autoregressive relay

Over 100 generated tokens (5 prompts $\times$ 20 steps), the relay obtains 47% tri-consistency, 15% B-only matches, 5% A-only matches, and 33% tokens that match neither baseline (Table 2). The 62% aggregate alignment with B is consistent with the early split $K = 6$, where most layers come from the receiver.

Table 3 shows autoregressive teacher-forcing PPL on three evaluation texts. The relay mean ratio to Instruct is 0.949. It improves over Instruct on all three texts, but on two Chinese texts it remains worse than the Base checkpoint, which limits the strength of the claim.

6 Analysis

Three-zone split behavior. Layerwise hidden-state divergence for Base $\rightarrow$ Instruct shows a stable early zone, a transition zone, and a late divergence zone. In our logs, layers 0–1 have $\delta_i < 0.20$ and cosine similarity above 0.98; layers 2–15 have $\delta_i \in [0.53, 1.06]$ and cosine similarity in $[0.96, 0.99]$;

Table 3: Autoregressive teacher-forcing PPL at split $K = 6$. The result supports short-horizon feasibility but should not be interpreted as benchmark-level superiority.

Text	PPL _A	PPL _B	PPL _R	vs. best	vs. B
Quantum computing (ZH)	2.84	3.33	3.17	1.118	0.953
Machine learning (EN)	2.34	2.14	2.11	0.985	0.985
Deep learning (ZH)	6.68	8.59	7.82	1.171	0.910
<i>Mean</i>				1.091	0.949

Text	A	B	Relay
Quantum (ZH)	2.84	3.33	3.17
Machine learning	2.34	2.14	2.11
Deep learning	6.68	8.59	7.82

Figure 2: Autoregressive teacher-forcing PPL. Relay is consistently below the Instruct baseline in this small evaluation, but not consistently below the best baseline.

128 layers 16–23 have $\delta_i \in [2.0, 4.5]$ and cosine similarity near 0.96. The primary split $K = 6$ lies near
 129 the early transition region.

130 **Interpretation of neither tokens.** The 33% neither category does not by itself imply failure: man-
 131 ual inspection found grammatical and semantically coherent continuations. Nevertheless, because
 132 manual inspection is subjective and short-horizon, the current result should be treated as qualitative
 133 support only. Longer generations and task-level metrics are needed.

134 **Why relay can help.** One possible explanation is that the Base checkpoint preserves early feature
 135 representations that are useful for next-token likelihood, while later Instruct layers impose response
 136 style and instruction-following behavior. Latent relay may combine these advantages when split
 137 early. This remains a hypothesis; controlled ablations over multiple checkpoints and fine-tuning
 138 strengths are necessary.

139 7 Limitations and responsible release

140 **Architecture and checkpoint scope.** The method assumes equal hidden size, layer layout, tok-
 141 enizer, vocabulary, and RoPE configuration. Cross-architecture relay would require a learned pro-
 142 jection or adapter and is not evaluated here.

143 **Scale and length.** All reported experiments use one 2B-parameter checkpoint pair, CPU float32
 144 inference, five prompts, and 20-token generation. Results may change at 7B+ scale, with quantiza-
 145 tion, with GPU kernels, or with longer contexts where error accumulation can become visible.

146 **Evaluation strength.** PPL and token consistency are proxies. We do not report MMLU, GSM8K,
 147 HumanEval, safety, calibration, or human-preference evaluations. No statistical significance claim
 148 is made.

149 **Assets and reproducibility.** The paper depends on existing model checkpoints and local evalu-
 150 ation code. For NeurIPS review, an anonymized supplementary ZIP should include the relay im-
 151 plementation, environment file, exact model identifiers and licenses, exact prompts, and scripts that
 152 reproduce Tables 1–3. If model licenses restrict redistribution, the supplement should give deter-
 153 ministic download instructions rather than repackaging model weights.

154 **Dual use and safeguards.** Latent relay can lower the memory cost of combining model capabili-
 155 ties, which may be beneficial for efficient research but could also make multi-checkpoint generation
 156 systems easier to deploy on constrained hardware. The release should avoid unsafe prompt exam-
 157 ples, follow the licenses and use policies of the underlying checkpoints, and document that relay
 158 does not add safety guarantees beyond those of the receiver model.

8 Conclusion

Latent relay is a same-architecture activation handoff for checkpoint chaining. In a Qwen3.5-2B Base $\rightarrow$ Instruct case study, it is exact when the checkpoints are weight-identical and remains stable in short cross-checkpoint generation. The empirical evidence is preliminary, but it suggests that same-family checkpoints can share enough latent structure to support low-memory expert composition without token-space mediation. The next step is to replace the present feasibility study with broader benchmark evaluation, longer-context stress tests, adapter-based cross-architecture relays, and an anonymized reproducibility package.

References

- Bansal, Y., Nakkiran, P., and Barak, B. (2021). Revisiting model similarity in language models. *arXiv:2104.00154*.
- Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23(120):1–39.
- Goddard, C., et al. (2024). Arcee’s MergeKit: A toolkit for merging large language models. *arXiv:2403.13257*.
- Lenc, K. and Vedaldi, A. (2015). Understanding image representations by measuring their equivariance and equivalence. *CVPR*.
- Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. *ICML*.
- Shazeer, N., et al. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*.
- Yadav, P., et al. (2023). TIES-merging: Resolving interference when merging models. *NeurIPS*.

NeurIPS Paper Checklist

1. Claims. **Question:** Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope? **Answer:** [Yes]. **Justification:** The abstract and introduction frame the work as a feasibility study and explicitly limit claims to same-architecture checkpoints and short-horizon experiments.

2. Limitations. **Question:** Does the paper discuss the limitations of the work performed by the authors? **Answer:** [Yes]. **Justification:** Section 7 discusses architecture scope, scale, context length, evaluation strength, assets, and safeguards.

3. Theory assumptions and proofs. **Question:** For each theoretical result, does the paper provide the full set of assumptions and a complete proof? **Answer:** [Yes]. **Justification:** Proposition 1 states the required same-architecture and fixed-RoPE assumptions and provides a short proof.

4. Experimental result reproducibility. **Question:** Does the paper fully disclose all information needed to reproduce the main experimental results to the extent that it affects the claims? **Answer:** [No]. **Justification:** The paper specifies the protocol and hardware, but the final anonymized supplementary package should still include exact prompts, model identifiers, environment files, and scripts.

5. Open access to data and code. **Question:** Does the paper provide open access to the data and code, with sufficient instructions to reproduce the main experimental results? **Answer:** [No]. **Justification:** Code is not included in this manuscript file; the intended NeurIPS submission should add an anonymized ZIP with code and instructions.

201 **6. Experimental setting/details.** **Question:** Does the paper specify all training and test details
 202 necessary to understand the results? **Answer:** [Yes]. **Justification:** The paper reports the model
 203 configuration, precision, hardware, split point, perplexity metric, token-consistency metric, and eval-
 204 uation scope; no training is performed.

205 **7. Experiment statistical significance.** **Question:** Does the paper report error bars or other ap-
 206 propriate information about statistical significance? **Answer:** [No]. **Justification:** The experiments
 207 are deterministic and small-scale; Section 4 explicitly states that no statistical-significance claim is
 208 made.

209 **8. Experiments compute resources.** **Question:** For each experiment, does the paper provide
 210 sufficient information on compute resources? **Answer:** [Yes]. **Justification:** Section 4 reports
 211 CPU, memory, storage, GPU non-use, precision, and subprocess isolation. Wall-clock time should
 212 be added to the supplement if available.

213 **9. Code of ethics.** **Question:** Does the research conform with the NeurIPS Code of Ethics?
 214 **Answer:** [Yes]. **Justification:** The work uses existing language-model checkpoints and synthetic
 215 prompts, does not involve human subjects, and discusses possible dual-use considerations.

216 **10. Broader impacts.** **Question:** Does the paper discuss potential positive and negative societal
 217 impacts? **Answer:** [Yes]. **Justification:** Section 7 discusses efficiency benefits and possible misuse
 218 from lowering memory barriers for multi-checkpoint generation systems.

219 **11. Safeguards.** **Question:** Does the paper describe safeguards for responsible release of data or
 220 models with high misuse risk? **Answer:** [Yes]. **Justification:** Section 7 recommends anonymized
 221 code release, adherence to underlying model licenses and use policies, and documentation that relay
 222 adds no safety guarantees.

223 **12. Licenses for existing assets.** **Question:** Are existing assets credited and are their licenses and
 224 terms of use mentioned and respected? **Answer:** [No]. **Justification:** The manuscript identifies the
 225 checkpoint family but the exact model identifiers and license terms still need to be included in the
 226 anonymized supplement or final paper.

227 **13. New assets.** **Question:** Are new assets introduced in the paper well documented? **Answer:**
 228 [N/A]. **Justification:** The paper does not introduce a new dataset or pretrained model; it proposes
 229 an inference-time mechanism and evaluation scripts.

230 **14. Crowdsourcing and human subjects.** **Question:** For crowdsourcing experiments and
 231 human-subjects research, does the paper include instructions, screenshots, and compensation de-
 232 tails? **Answer:** [N/A]. **Justification:** The paper does not involve crowdsourcing or human-subjects
 233 research.

234 **15. IRB approvals.** **Question:** Does the paper describe risks to study participants and IRB ap-
 235 provals or equivalent review? **Answer:** [N/A]. **Justification:** The paper does not involve human
 236 participants.

237 **16. Declaration of LLM usage.** **Question:** Does the paper describe LLM usage if it is an impor-
 238 tant, original, or non-standard component of the core methods? **Answer:** [Yes]. **Justification:** The
 239 core method directly manipulates hidden states of language-model checkpoints, and the paper states
 240 the exact role of these models in the method and experiments.